- Bill reacts to the problem.

The same thing with specifics attached:

- A Button decides it wants an ActionListener.
- A Button is pressed--an event happens:
- The ActionListener notices the Button was pressed.
- The Listener reacts to the event.
- Fully described in Java Terms:
- A Button adds an ActionListener.
- A Button is pressed--an event happens:
- The method actionPerformed is executed, receiving an ActionEvent object.
- The Listener reacts to the event.

An event source is an object that can register listener objects and send those listeners event objects. In practice, the event source can send out event objects to all registered listeners when that event occurs. The listener object(s) will then use the details in the event object to decide how to react to the event. A "Listener Object" [an instance of a class] implements a special interface called a "listener interface." When you ask the listener object to pay attention to your event source, that is called registering the listener object with the source object. You do that with the following lines of code:

```
eventSourceObject.addEventListener( eventListenerObject
);
```

Every event handler requires 3 bits of code:

Code that says the class implements a listener interface:

```
public class MyClass implements ActionListener
```

Code that registers a listener on one or more components.

```
someComponent.addActionListener( MyClass );
```

Code that implements the methods in the listener interface.

```
public void actionPerformed(ActionEvent e)
{
...//code that reacts to the action...
}
```

Lets take another Example:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class Craps extends JApplet implements
```